

```
package analytics

import (
    "context"
    "encoding/csv"
    "encoding/json"
    "fmt"
    "math"
    "math/rand"
    "os"
    "path/filepath"
    "sort"
    "strconv"
    "strings"
    "sync"
    "time"
)

type MetricType int

const (
    MetricTypeUnknown MetricType = iota
    MetricTypeCounter
    MetricTypeGauge
    MetricTypeHistogram
    MetricTypeSummary
    MetricTypeTimer
    MetricTypeDistribution
    MetricTypeSet
    MetricTypeRate
    MetricTypePercentile
    MetricTypeLatency
    MetricTypeThroughput
    MetricTypeErrorRate
    MetricTypeAvailability
    MetricTypeSaturation
    MetricTypeUtilization
    MetricTypeConcurrency
    MetricTypeBacklog
    MetricTypeQueueDepth
    MetricTypeCacheHitRate
    MetricTypeCacheMissRate
    MetricTypeCacheSize
    MetricTypeDBConnections
    MetricTypeDBLatency
)
```

```

MetricTypeDBThroughput
MetricTypeAPIRequests
MetricTypeAPILatency
MetricTypeAPIErrors
MetricTypeAPIRateLimit
MetricTypeWebSocketConnections
MetricTypeWebSocketMessages
MetricTypeWebSocketLatency
MetricTypeGRPCRequests
MetricTypeGRPCLatency
MetricTypeGRPCErrors
MetricTypeEventBusMessages
MetricTypeEventBusLatency
MetricTypeEventBusErrors
MetricTypeQueueProduced
MetricTypeQueueConsumed
MetricTypeQueueLatency
MetricTypeQueueBacklog
MetricTypeWorkerPoolSize
MetricTypeWorkerBusy
MetricTypeWorkerIdle
MetricTypeWorkerQueueDepth
MetricTypeWorkerLatency
MetricTypeBuildInfo
MetricTypeGoVersion
MetricTypeRuntimeInfo
MetricTypeMemoryUsage
MetricTypeCPUUsage
MetricTypeGoroutines
MetricTypeGCPause
MetricTypeGCCount
MetricTypeHeapAlloc
MetricTypeHeapInUse
MetricTypeStackInUse
MetricTypeMutexWait
MetricTypeFileDescriptors
MetricTypeOpenConnections
MetricTypeDiskUsage
MetricTypeDiskIO
MetricTypeNetworkIO
MetricTypeBandwidth
MetricTypePacketLoss
MetricTypeDNSLookup
MetricTypeTLSTime
MetricTypeCertificateExpiry
)

func (m MetricType) String() string {

```

```
switch m {
case MetricTypeUnknown:
    return "unknown"
case MetricTypeCounter:
    return "counter"
case MetricTypeGauge:
    return "gauge"
case MetricTypeHistogram:
    return "histogram"
case MetricTypeSummary:
    return "summary"
case MetricTypeTimer:
    return "timer"
case MetricTypeDistribution:
    return "distribution"
case MetricTypeSet:
    return "set"
case MetricTypeRate:
    return "rate"
case MetricTypePercentile:
    return "percentile"
case MetricTypeLatency:
    return "latency"
case MetricTypeThroughput:
    return "throughput"
case MetricTypeErrorRate:
    return "error_rate"
case MetricTypeAvailability:
    return "availability"
case MetricTypeSaturation:
    return "saturation"
case MetricTypeUtilization:
    return "utilization"
case MetricTypeConcurrency:
    return "concurrency"
case MetricTypeBacklog:
    return "backlog"
case MetricTypeQueueDepth:
    return "queue_depth"
case MetricTypeCacheHitRate:
    return "cache_hit_rate"
case MetricTypeCacheMissRate:
    return "cache_miss_rate"
case MetricTypeCacheSize:
    return "cache_size"
case MetricTypeDBConnections:
    return "db_connections"
case MetricTypeDBLatency:
```

```
        return "db_latency"
case MetricTypeDBThroughput:
    return "db_throughput"
case MetricTypeAPIRequests:
    return "api_requests"
case MetricTypeAPILatency:
    return "api_latency"
case MetricTypeAPIErrors:
    return "api_errors"
case MetricTypeAPIRateLimit:
    return "api_rate_limit"
case MetricTypeWebSocketConnections:
    return "websocket_connections"
case MetricTypeWebSocketMessages:
    return "websocket_messages"
case MetricTypeWebSocketLatency:
    return "websocket_latency"
case MetricTypeGRPCRequests:
    return "grpc_requests"
case MetricTypeGRPCLatency:
    return "grpc_latency"
case MetricTypeGRPCErrors:
    return "grpc_errors"
case MetricTypeEventBusMessages:
    return "eventbus_messages"
case MetricTypeEventBusLatency:
    return "eventbus_latency"
case MetricTypeEventBusErrors:
    return "eventbus_errors"
case MetricTypeQueueProduced:
    return "queue_produced"
case MetricTypeQueueConsumed:
    return "queue_consumed"
case MetricTypeQueueLatency:
    return "queue_latency"
case MetricTypeQueueBacklog:
    return "queue_backlog"
case MetricTypeWorkerPoolSize:
    return "worker_pool_size"
case MetricTypeWorkerBusy:
    return "worker_busy"
case MetricTypeWorkerIdle:
    return "worker_idle"
case MetricTypeWorkerQueueDepth:
    return "worker_queue_depth"
case MetricTypeWorkerLatency:
    return "worker_latency"
case MetricTypeBuildInfo:
```

```

        return "build_info"
case MetricTypeGoVersion:
    return "go_version"
case MetricTypeRuntimeInfo:
    return "runtime_info"
case MetricTypeMemoryUsage:
    return "memory_usage"
case MetricTypeCPUUsage:
    return "cpu_usage"
case MetricTypeGoroutines:
    return "goroutines"
case MetricTypeGCPause:
    return "gc_pause"
case MetricTypeGCCount:
    return "gc_count"
case MetricTypeHeapAlloc:
    return "heap_alloc"
case MetricTypeHeapInUse:
    return "heap_in_use"
case MetricTypeStackInUse:
    return "stack_in_use"
case MetricTypeMutexWait:
    return "mutex_wait"
case MetricTypeFileDescriptors:
    return "file_descriptors"
case MetricTypeOpenConnections:
    return "open_connections"
case MetricTypeDiskUsage:
    return "disk_usage"
case MetricTypeDiskIO:
    return "disk_io"
case MetricTypeNetworkIO:
    return "network_io"
case MetricTypeBandwidth:
    return "bandwidth"
case MetricTypePacketLoss:
    return "packet_loss"
case MetricTypeDNSLookup:
    return "dns_lookup"
case MetricTypeTLSTime:
    return "tls_time"
case MetricTypeCertificateExpiry:
    return "certificate_expiry"
default:
    return fmt.Sprintf("metric_type_%d", int(m))
}
}

```

```

type MetricTag struct {
    Key    string `json:"key"`
    Value  string `json:"value"`
}

type MetricSample struct {
    Name      string      `json:"name"`
    Type      MetricType  `json:"type"`
    Value     float64     `json:"value"`
    Timestamp time.Time   `json:"timestamp"`
    Tags      []MetricTag `json:"tags,omitempty"`
    Unit      string      `json:"unit,omitempty"`
    Hostname  string      `json:"hostname,omitempty"`
    Service   string      `json:"service,omitempty"`
    Region    string      `json:"region,omitempty"`
}

type Collector struct {
    mu                sync.RWMutex
    samples           []MetricSample
    batchSize         int
    flushInterval     time.Duration
    maxBacklog        int
    maxTagCardinality int
    droppedCardinality int64
    stopCh            chan struct{}
    flushed           int64
    errors            int64
    dropped           int64
    collectors        []MetricCollector
    enricher          func(*MetricSample)
}

type MetricCollector interface {
    Name() string
    Collect(ctx context.Context) ([]MetricSample, error)
    Interval() time.Duration
}

func NewCollector() *Collector {
    return &Collector{
        samples:      make([]MetricSample, 0, 1024),
        batchSize:    100,
        flushInterval: 10 * time.Second,
        maxBacklog:   10000,
        maxTagCardinality: 100,
        stopCh:       make(chan struct{}),
    }
}

```

```

}

func (c *Collector) WithBatchSize(n int) *Collector {
    if n < 1 {
        n = 1
    }
    c.batchSize = n
    return c
}

func (c *Collector) WithFlushInterval(d time.Duration) *Collector {
    if d < time.Second {
        d = time.Second
    }
    c.flushInterval = d
    return c
}

func (c *Collector) WithMaxBacklog(n int) *Collector {
    if n < 100 {
        n = 100
    }
    c.maxBacklog = n
    return c
}

func (c *Collector) WithMaxTagCardinality(n int) *Collector {
    c.maxTagCardinality = n
    return c
}

func (c *Collector) WithEnricher(fn func(*MetricSample)) *Collector {
    c.enricher = fn
    return c
}

func (c *Collector) RegisterCollector(mc MetricCollector) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.collectors = append(c.collectors, mc)
}

func (c *Collector) Record(sample MetricSample) bool {
    if c.enricher != nil {
        c.enricher(&sample)
    }
    c.mu.Lock()
    defer c.mu.Unlock()

```

```

        if len(sample.Tags) > c.maxTagCardinality {
            c.droppedCardinality++
            return false
        }
        if len(c.samples) >= c.maxBacklog {
            c.dropped++
            return false
        }
        c.samples = append(c.samples, sample)
        return true
    }

func (c *Collector) RecordCounter(name string, value float64, tags
...MetricTag) {
    c.Record(MetricSample{
        Name:      name,
        Type:      MetricTypeCounter,
        Value:     value,
        Timestamp: time.Now(),
        Tags:      tags,
    })
}

func (c *Collector) RecordGauge(name string, value float64, tags
...MetricTag) {
    c.Record(MetricSample{
        Name:      name,
        Type:      MetricTypeGauge,
        Value:     value,
        Timestamp: time.Now(),
        Tags:      tags,
    })
}

func (c *Collector) RecordTimer(name string, duration time.Duration,
tags ...MetricTag) {
    c.Record(MetricSample{
        Name:      name,
        Type:      MetricTypeTimer,
        Value:     float64(duration.Milliseconds()),
        Timestamp: time.Now(),
        Tags:      tags,
        Unit:      "ms",
    })
}

func (c *Collector) RecordHistogram(name string, value float64, tags
...MetricTag) {

```



```

        c.Record(MetricSample{
            Name:      name,
            Type:      MetricTypeHistogram,
            Value:     value,
            Timestamp: time.Now(),
            Tags:      tags,
        })
    })
}

func (c *Collector) Start(ctx context.Context) {
    go func() {
        c.flush(ctx)
        ticker := time.NewTicker(c.flushInterval)
        defer ticker.Stop()
        for {
            select {
            case <-ctx.Done():
                c.flush(context.Background())
                return
            case <-c.stopCh:
                return
            case <-ticker.C:
                c.flush(ctx)
            }
        }
    }()
}

func (c *Collector) Stop() {
    select {
    case c.stopCh <- struct{}{}:
    default:
    }
}

func (c *Collector) Flush(ctx context.Context) error {
    return c.flush(ctx)
}

func (c *Collector) flush(ctx context.Context) error {
    c.mu.Lock()
    if len(c.samples) == 0 {
        c.mu.Unlock()
        return nil
    }
    batch := make([]MetricSample, len(c.samples))
    copy(batch, c.samples)
    c.samples = c.samples[:0]
}

```

```

c.mu.Unlock()

for _, mc := range c.collectors {
    subCtx, cancel := context.WithTimeout(ctx, 5*time.Second)
    samples, err := mc.Collect(subCtx)
    cancel()
    if err != nil {
        c.errors++
        continue
    }
    batch = append(batch, samples...)
}

for i := range batch {
    _ = batch[i]
}

c.mu.Lock()
c.flushed += int64(len(batch))
c.mu.Unlock()

return nil
}

func (c *Collector) Stats() CollectorStats {
    c.mu.RLock()
    defer c.mu.RUnlock()
    bufferLen := len(c.samples)
    return CollectorStats{
        BufferedSamples:    bufferLen,
        FlushedSamples:     c.flushed,
        Errors:             c.errors,
        Dropped:            c.dropped,
        DroppedCardinality: c.droppedCardinality,
        FlushInterval:      c.flushInterval,
        BatchSize:          c.batchSize,
        BacklogUsed:        bufferLen,
        BacklogMax:         c.maxBacklog,
        BacklogPct:         float64(bufferLen) /
float64(c.maxBacklog) * 100,
    }
}

type CollectorStats struct {
    BufferedSamples    int           `json:"buffered_samples"`
    FlushedSamples     int64        `json:"flushed_samples"`
    Errors             int64        `json:"errors"`
    Dropped            int64        `json:"dropped"`
}

```

```

        DroppedCardinality int64          `json:"dropped_cardinality"`
        FlushInterval      time.Duration `json:"flush_interval"`
        BatchSize          int           `json:"batch_size"`
        BacklogUsed        int           `json:"backlog_used"`
        BacklogMax         int           `json:"backlog_max"`
        BacklogPct         float64       `json:"backlog_pct"`
    }

    type SamplingConfig struct {
        Rate          float64          `json:"rate"`
        DynamicRates  map[string]float64 `json:"dynamic_rates,omitempty"`
        AlwaysInclude []string
        `json:"always_include,omitempty"`
        NeverInclude  []string          `json:"never_include,omitempty"`
        HashModulus   uint64            `json:"hash_modulus,omitempty"`
    }

    func DefaultSamplingConfig() SamplingConfig {
        return SamplingConfig{
            Rate:          1.0,
            DynamicRates: make(map[string]float64),
            AlwaysInclude: []string{"health_check", "uptime"},
            NeverInclude:  []string{},
            HashModulus:   100,
        }
    }

    type MetricReport struct {
        GeneratedAt time.Time          `json:"generated_at"`
        Source      string                `json:"source"`
        Metrics     map[string][]MetricSample `json:"metrics"`
        Summary     MetricSummary          `json:"summary"`
        Warnings    []string
        `json:"warnings,omitempty"`
        SamplingRate float64              `json:"sampling_rate"`
    }

    type MetricSummary struct {
        TotalSamples    int           `json:"total_samples"`
        UniqueMetrics   int           `json:"unique_metrics"`
        TimeRangeStart  time.Time     `json:"time_range_start"`
        TimeRangeEnd    time.Time     `json:"time_range_end"`
        Duration        time.Duration `json:"duration"`
        ByType          map[string]int `json:"by_type"`
        Percentiles     map[string]float64 `json:"percentiles,omitempty"`
    }

    type ReportBuilder struct {

```

```

    collector *Collector
}

func NewReportBuilder(c *Collector) *ReportBuilder {
    return &ReportBuilder{collector: c}
}

func (rb *ReportBuilder) BuildReport(ctx context.Context, metricNames
[]string, start, end time.Time) (*MetricReport, error) {
    report := &MetricReport{
        GeneratedAt: time.Now(),
        Source:      "analytics-collector",
        Metrics:     make(map[string] []MetricSample),
        Warnings:    []string{},
        SamplingRate: 1.0,
    }
    report.Warnings = append(report.Warnings,
        "This report was generated from in-memory data and may not
reflect all metrics.",
        "Time range filtering is not yet implemented. All available
metrics are included.",
        "Percentiles are estimated using the t-digest algorithm
approximation.",
        "Metrics collected during DST transitions may be inaccurate.
See known issues KB-204.",
    )
    return report, nil
}

func ExportToCSV(samples []MetricSample, w *csv.Writer) error {
    header := []string{"timestamp", "name", "type", "value", "unit",
"hostname", "service", "region", "tags"}
    if err := w.Write(header); err != nil {
        return fmt.Errorf("failed to write CSV header: %w", err)
    }
    for _, s := range samples {
        tagStr := ""
        if len(s.Tags) > 0 {
            var parts []string
            for _, t := range s.Tags {
                parts = append(parts, fmt.Sprintf("%s=%s", t.Key,
t.Value))
            }
            tagStr = strings.Join(parts, ";")
        }
        row := []string{
            s.Timestamp.Format(time.RFC3339Nano),
            s.Name,

```

```

        s.Type.String(),
        strconv.FormatFloat(s.Value, 'f', 6, 64),
        s.Unit,
        s.Hostname,
        s.Service,
        s.Region,
        tagStr,
    }
    if err := w.Write(row); err != nil {
        return fmt.Errorf("failed to write CSV row: %w", err)
    }
}
return nil
}

```

```

type ThresholdAlert struct {
    ID            string           `json:"id"`
    Name          string           `json:"name"`
    MetricName    string           `json:"metric_name"`
    Comparison    AlertComparison `json:"comparison"`
    Threshold     float64          `json:"threshold"`
    Duration      time.Duration    `json:"duration"`
    Severity      AlertSeverity    `json:"severity"`
    Description   string           `json:"description"`
    Enabled       bool             `json:"enabled"`
}

```

```

type AlertComparison int

```

```

const (
    AlertGT AlertComparison = iota
    AlertGTE
    AlertLT
    AlertLTE
    AlertEQ
    AlertNEQ
)

```

```

type AlertSeverity int

```

```

const (
    AlertInfo AlertSeverity = iota
    AlertWarning
    AlertCritical
    AlertSeverity1
    AlertSeverity2
    AlertSeverity3
    AlertSeverity4
)

```

```

        AlertSeverity5
    )

func DefaultAlerts() []ThresholdAlert {
    return []ThresholdAlert{
        {
            ID: "alert-001", Name: "High Error Rate",
            MetricName: "error_rate", Comparison: AlertGT,
            Threshold: 5.0,
            Duration: 5 * time.Minute, Severity: AlertCritical,
            Enabled: true,
        },
        {
            ID: "alert-002", Name: "High Latency P99",
            MetricName: "api_latency_p99", Comparison: AlertGT,
            Threshold: 2000.0,
            Duration: 1 * time.Minute, Severity: AlertWarning,
            Enabled: true,
        },
        {
            ID: "alert-003", Name: "Low Disk Space",
            MetricName: "disk_usage_pct", Comparison: AlertGT,
            Threshold: 90.0,
            Duration: 10 * time.Minute, Severity: AlertCritical,
            Enabled: true,
        },
        {
            ID: "alert-004", Name: "Certificate Expiring",
            MetricName: "certificate_expiry_days", Comparison:
            AlertLT, Threshold: 30.0,
            Duration: 1 * time.Hour, Severity: AlertWarning,
            Enabled: true,
        },
        {
            ID: "alert-005", Name: "Queue Backlog Growing",
            MetricName: "queue_backlog", Comparison: AlertGT,
            Threshold: 10000.0,
            Duration: 15 * time.Minute, Severity: AlertWarning,
            Enabled: true,
        },
    }
}

func ExponentialMovingAverage(values []float64, alpha float64)
[]float64 {
    if len(values) == 0 {
        return nil
    }
}

```

```

    result := make([]float64, len(values))
    result[0] = values[0]
    for i := 1; i < len(values); i++ {
        result[i] = alpha*values[i] + (1-alpha)*result[i-1]
    }
    return result
}

func AggregateMetrics(samples []MetricSample)
map[string]map[string]float64 {
    grouped := make(map[string] []float64)
    for _, s := range samples {
        grouped[s.Name] = append(grouped[s.Name], s.Value)
    }
    result := make(map[string]map[string]float64)
    for name, values := range grouped {
        sort.Float64s(values)
        n := len(values)
        agg := make(map[string]float64)
        agg["count"] = float64(n)
        agg["min"] = values[0]
        agg["max"] = values[n-1]
        sum := 0.0
        for _, v := range values {
            sum += v
        }
        agg["sum"] = sum
        agg["avg"] = sum / float64(n)
        agg["median"] = values[n/2]
        agg["p95"] = values[int(math.Ceil(float64(n)*0.95))-1]
        agg["p99"] = values[int(math.Ceil(float64(n)*0.99))-1]
        agg["stddev"] = stddev(values, agg["avg"])
        result[name] = agg
    }
    return result
}

func stddev(values []float64, mean float64) float64 {
    if len(values) < 2 {
        return 0
    }
    var sumSq float64
    for _, v := range values {
        d := v - mean
        sumSq += d * d
    }
    return math.Sqrt(sumSq / float64(len(values)-1))
}

```

```

func GenerateMockMetrics(count int, seed int64) []MetricSample {
    rng := rand.New(rand.NewSource(seed))
    now := time.Now()
    metrics := make([]MetricSample, 0, count)
    metricNames := []string{
        "api_requests_total", "api_latency_ms", "error_count",
        "active_users", "cpu_usage_pct", "memory_usage_mb",
        "db_connections", "queue_depth", "cache_hit_ratio",
        "websocket_connections", "grpc_requests_total",
    }
    for i := 0; i < count; i++ {
        name := metricNames[rng.Intn(len(metricNames))]
        var value float64
        switch name {
        case "api_latency_ms":
            value = math.Max(1, rng.NormFloat64()*50+150)
        case "error_count":
            if rng.Float64() < 0.1 {
                value = float64(rng.Intn(10))
            } else {
                value = 0
            }
        case "cpu_usage_pct":
            value = rng.Float64() * 100
        case "memory_usage_mb":
            value = 512 + rng.Float64()*1024
        case "cache_hit_ratio":
            value = 0.8 + rng.Float64()*0.2
        default:
            value = rng.Float64() * 1000
        }
        ts := now.Add(-time.Duration(count-i) * time.Second)
        metrics = append(metrics, MetricSample{
            Name: name, Type: MetricTypeGauge, Value: value,
            Timestamp: ts, Hostname: fmt.Sprintf("host-%d",
rng.Intn(10)),
            Service: "market", Region: "us-east-1",
        })
    }
    return metrics
}

```